

Métodos Numéricos I – Tarea 9

Pedro D. Llerenas

pedro.llerenas@cimat.mx

2025-11-02

1. Consideremos la función $\sin(x)$:

- a) Genera un código basado en la interpolación de Hermite con diferencias divididas visto en clase.
b) Con los valores de la siguiente tabla, estima el valor de $\sin(0.34)$ utilizando interpolación de Hermite con diferencias divididas. Halla el error absoluto de la aproximación con el valor correcto.

x	$\sin x$	$D_x \sin x = \cos x$
0.30	0.29552	0.95534
0.32	0.31457	0.94924
0.35	0.34290	0.93937

- c) A la tabla añade $\sin(0.33) = 0.32404$ y $\cos(0.33) = 0.94604$ y recalcula el valor de $\sin(0.34)$. Halla el error absoluto, ¿qué puedes concluir?

b)

Dada una función $f \in C^1[a, b]$ y los puntos $x_0, \dots, x_n \in [a, b]$ todos distintos, el polinomio único de menor grado que concide con f y f' en los puntos $x_0, \dots, x_n$ es el polinomio de Hermite de grado a lo más $2n + 1$ dado por

$$H_{2n+1}(x) = \sum_{j=0}^n f(x_j) H_{n,j}(x) + \sum_{j=0}^n f'(x_j) \hat{H}_{n,j}(x)$$

donde

$$H_{n,j}(x) = [1 - 2(x - x_j)L'_{n,j}(x_j)] L_{n,j}^2(x)$$

$$\hat{H}_{n,j}(x) = (x - x_j) L_{n,j}^2(x),$$

y $L_{n,j}(x)$ es el j —ésimo coeficiente del polinomio de Lagrange de grado n .

Si $f \in C^{2n+2}[a, b]$, entonces

$$f(x) = H_{2n+1}(x) + \frac{(x - x_0)^2 \cdots (x - x_n)^2}{(2n + 2)!} f^{(2n+2)}(\xi(x)),$$

para $\xi(x)$ en el intervalo (a, b) .

Podemos reducir la complejidad de construcción de dichos polinomios con el uso de diferencias divididas. Definimos la sucesión $\{z_i\}_{i=0}^{2n+1}$, dada por

$$z_{2i} = z_{2i+1} = x_i, \quad \forall i = 0, \dots, n.$$

Construimos el polinomio de Hermite por

$$H_{2n+1}(x) = f[z_0] + \sum_{k=1}^{2n+1} f[z_0, \dots, z_k] (x - z_0)(x - z_1) \cdots (x - z_{k-1})$$

Para el ejemplo dado, queremos interpolar el valor $x = 0.34$ para $f(x) = \sin(x)$. Para ejecutar el programa, usamos

```
> make run-p1 ARGS="0.34 ex/hermite/x.txt ex/hermite/fx.txt ex/hermite/dfx.txt"
```

Esto nos imprime en la consola

```
> f(0.34) = 0.33348889007407406  
> sin(0.34) = 0.3334870921408144  
> abs_err = 1.7979332596618214e-06
```

Además, nos imprime los correspondientes coeficientes del polinomio generado, en el archivo `ex/hermite/coeff.csv`.

c)

Ahora, si incluimos el punto adicional, tenemos la tabla

x	$\sin x$	$D_x \sin x = \cos x$
0.30	0.29552	0.95534
0.32	0.31457	0.94924
0.33	0.32404	0.94604
0.35	0.34290	0.93937

Para correr nuestro programa con este dato adicional, usamos

```
> make run-p1 ARGS="0.34 ex/hermite/x2.txt ex/hermite/fx2.txt ex/hermite/dfx2.txt"
```

Esto nos imprime en la consola

```
> f(0.34) = 0.3334977765925926  
> sin(0.34) = 0.3334870921408144  
> abs_err = 1.0684451778208981e-05
```

Además, nos imprime los correspondientes coeficientes del polinomio generado, en el archivo `ex/hermite/coeff.csv`.

Observamos que el error absoluto es mayor a cuando tenemos solamente 3 puntos. Si graficamos los polinomios dados, obtenemos las siguientes gráficas:

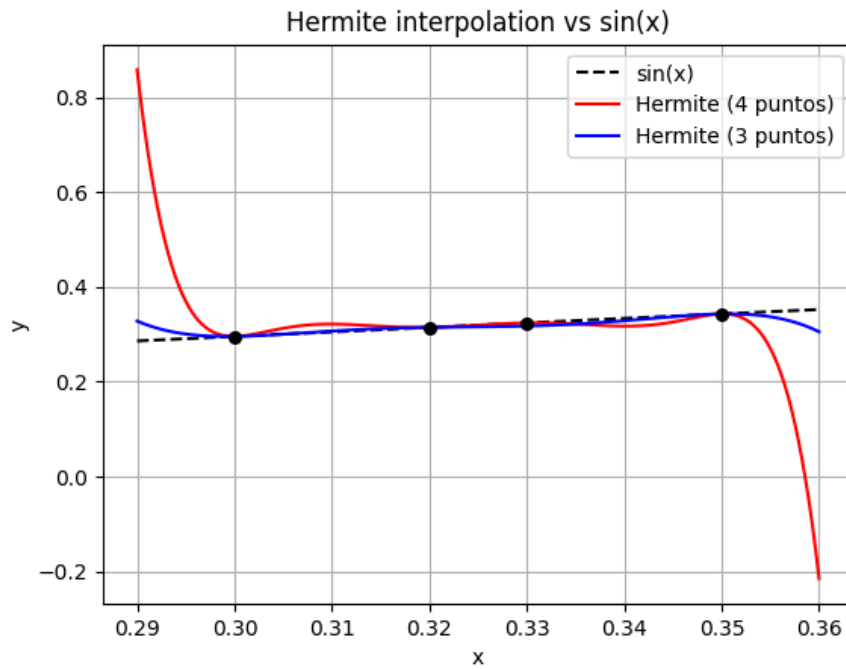


Figure 1: Ambos polinomios de Hermite comparados con $\sin x$

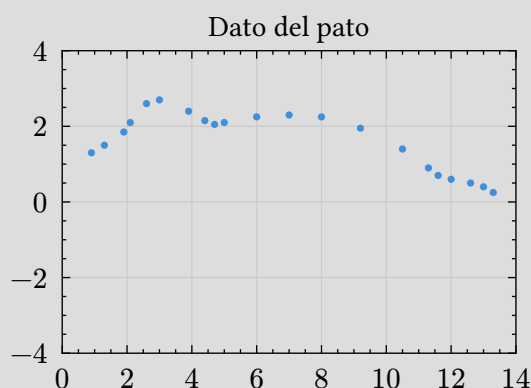
Como podemos observar, la adición del punto $x = 0.33$ causa que el polinomio generado por el algoritmo oscile alrededor del punto $x = 0.34$. Si observamos los coeficientes de los polinomios generados,

Hermite 4 puntos	Hermite 3 puntos
0.29552	0.29552
0.95534	0.95534
-0.142	-0.142
-1.05	-1.05
-32.7778	20.7778
17574.1	-436.296
-744815	NA
$2.94556e + 07$	NA

En particular, observamos los últimos coeficientes, que hacen que la curva que pasa por el punto $x = 0.34$ oscile demasiado, causando un mayor error absoluto.

2. Genera un código que realice interpolación cúbica natural. Puedes basarte en el algoritmo visto en clase. Con este algoritmo, recrea la parte superior de la imagen del pato dado por los puntos de la tabla:

x	0.9	1.3	1.9	2.1	2.6	3.0	3.9	4.4	4.7	5.0	6.0	7.0	8.0	9.2	10.5	11.3	11.6	12.0	12.6	13.0	13.3
$f(x)$	1.3	1.5	1.85	2.1	2.6	2.7	2.4	2.15	2.05	2.1	2.25	2.3	2.25	1.95	1.4	0.9	0.7	0.6	0.5	0.4	0.25



Nota: Para la interpolación puedes utilizar secciones o quitar puntos. La idea es que se acerque a la forma adecuada. Sólo debes explicar tu procedimiento en el reporte. Si alguno desea recrear al pato completo se le puede hacer llegar los puntos a interpolar.

Como se observó en el problema anterior, la interpolación por medio de polinomios de orden superior pueden oscilar de forma errática; es decir, pequeños cambios en pequeños intervalos pueden causar que el polinomio generado tenga fluctuaciones enormes sobre todo el rango.

Alternativamente, se busca interpolar la curva por tramos, donde cada subintervalo requiere de una nueva función de aproximación. La **interpolación de spline cúbico** es aquella que aproxima la función por tramos mediante el uso de polinomios cúbicos. Esto nos garantiza que la función de aproximación es continuamente diferenciable, y sus segundas derivadas son continuas.

Dada una función $f : [a, b] \rightarrow \mathbb{R}$ y un conjunto de nodos $a = x_0 < \dots < x_n = b$, un **interpolante de spline cúbico** S para f es una función que satisface las siguientes condiciones:

- a) $S(x)$ es un polinomio cúbico que se denota $S_j(x)$ en el subintervalo $[x_j, x_{j+1}]$ para cada $j = 0, 1, \dots, n-1$;
- b) $S_j(x_j) = f(x_j)$ y $S_j(x_{j+1}) = f(x_{j+1})$ para cada $j = 0, 1, \dots, n-1$;
- c) $S_{j+1}(x_{j+1}) = S_j(x_{j+1})$ para cada $j = 0, 1, \dots, n-2$;
- d) $S_{j+1}'(x_{j+1}) = S_j'(x_{j+1})$ para cada $j = 0, 1, \dots, n-2$;
- d) $S_{j+1}''(x_{j+1}) = S_j''(x_{j+1})$ para cada $j = 0, 1, \dots, n-2$;
- e) Uno de los siguientes conjuntos de condiciones de frontera se satisface:
 - i) $S''(x_0) = S''(x_n) = 0$ (**frontera natural**);
 - ii) $S'(x_0) = f'(x_0)$ y $S'(x_n) = f'(x_n)$ (**frontera condicionada**).

Para este ejercicio, usaremos la frontera natural.

Ejecutamos nuestro programa con

```
> make run-p2 ARGS="ex/spline-nat/x.txt ex/spline-nat/fx.txt"
```

Esto nos genera la tabla `ex/spline-nat/table.csv`, que contiene los datos de los polinomios interpoladores de cada subintervalo. Se ve de la siguiente manera:

i	x_i	a_i	b_i	c_i	d_i
0	0.9	1.3	0.539624	0	-0.247649
1	1.3	1.5	0.420752	-0.297179	0.946912
2	1.9	1.85	1.0868	1.40726	-2.95638
3	2.1	2.1	1.29494	-0.366567	-0.446635
4	2.6	2.6	0.593399	-1.03652	0.445051
5	3	2.7	-0.0221911	-0.502457	0.17416
6	3.9	2.4	-0.503406	-0.0322258	0.0780757
7	4.4	2.15	-0.477075	0.0848877	1.31417
8	4.7	2.05	-0.0713162	1.26764	-1.58122
9	5	2.1	0.26234	-0.155455	0.0431153
10	6	2.25	0.0807755	-0.0261092	-0.00466634
11	7	2.3	0.0145582	-0.0401082	-0.02445
12	8	2.25	-0.139008	-0.113458	0.0174707
13	9.2	1.95	-0.335834	-0.0505636	-0.0127279
14	10.5	1.4	-0.53183	-0.100202	-0.0203252
15	11.3	0.9	-0.731178	-0.148983	1.21341
16	11.6	0.7	-0.492949	0.943082	-0.839275
17	12	0.6	-0.141335	-0.0640482	0.0363821
18	12.6	0.5	-0.1789	0.00143957	-0.447971
19	13	0.4	-0.392775	-0.536126	0.595695

Table 1: Coeficientes del spline cúbico natural (pato)

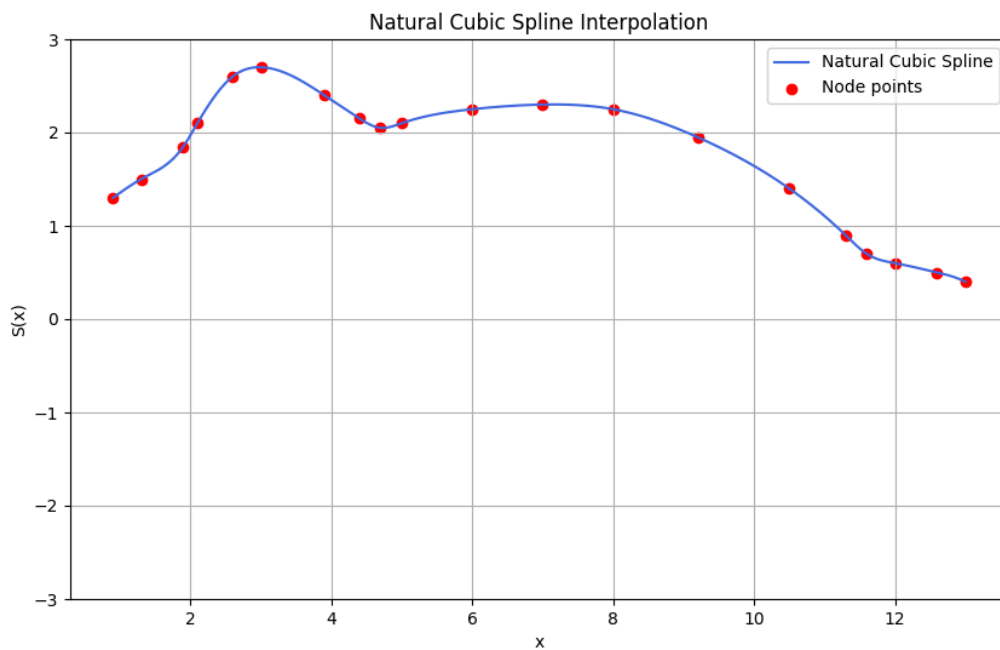


Figure 2: Spline cúbico que sigue la forma del pato

3. Genera un código que realice interpolación cúbica fijo o sujeto, puedes basarte en el algoritmo visto en clase. Con este algoritmo recrea la parte superior de Snoopy, dados los puntos de la tabla.

i	x_i	$f(x_i)$	$f'(x_i)$
0	1	3.0	1.0
1	2	3.7	
2	5	3.9	
3	6	4.2	
4	7	5.7	
5	8	6.6	
6	10	7.1	
7	13	6.7	
8	17	4.5	-0.67

Table 2: Curve 1

i	x_i	$f(x_i)$	$f'(x_i)$
0	17	4.5	3.0
1	20	7.0	
2	23	6.1	
3	24	5.6	
4	25	5.8	
5	27	5.2	
6	27.7	4.1	-0.4

Table 3: Curve 2

i	x_i	$f(x_i)$	$f'(x_i)$
0	27.7	4.1	0.33
1	28	4.3	
2	29	4.1	
3	30	3.0	-1.5

Table 4: Curve 3

Como explicamos en el segundo ejercicio, los splines cúbicos tienen la ventaja sobre los métodos que utilizan un solo polinomio para la interpolación de muchos puntos. En este caso, usaremos el método condicionado, que nos permite tener control sobre las derivadas en los puntos iniciales, lo que hace posible los cambios repentinos en la pendiente de cada función por sub-intervalo.

Para ejecutar el código correspondiente, hacemos 3 llamadas, una para cada función dada.

```
> make run-p3 ARGS="ex/spline-cond/x1.txt ex/spline-cond/fx1.txt 1.0 -0.67 ex/
spline-cond/table1.csv"
> make run-p3 ARGS="ex/spline-cond/x2.txt ex/spline-cond/fx2.txt 3.0 -4.0 ex/
spline-cond/table2.csv"
> make run-p3 ARGS="ex/spline-cond/x3.txt ex/spline-cond/fx3.txt 0.33 -1.5 ex/
spline-cond/table3.csv"
```

Cada ejecución nos genera una tabla `ex/spline-cond/table.csv` con los coeficientes correspondientes de cada uno de los polinomios interpoladores. Se ven de la siguiente manera:

i	x_i	a_i	b_i	c_i	d_i
0	1	3	1	-0.34681	0.04681
1	2	3.7	0.44681	-0.20638	0.0265552
2	5	3.9	-0.0744797	0.0326168	0.341863
3	6	4.2	1.01634	1.05821	-0.574548
4	7	5.7	1.40911	-0.665439	0.156329
5	8	6.6	0.54722	-0.19645	0.0239201
6	10	7.1	0.0484602	-0.0529297	-0.00255607
7	13	6.7	-0.338131	-0.0759343	0.00574178

Table 5: Coeficientes del spline cúbico condicionado para la Curva 1

i	x_i	a_i	b_i	c_i	d_i
0	17	4.5	3	-1.10071	0.126162
1	20	7	-0.197875	0.0347502	-0.0229307
2	23	6.1	-0.608501	-0.171626	0.280127
3	24	5.6	-0.111371	0.668756	-0.357385
4	25	5.8	0.153987	-0.403398	0.0882022
5	27	5.2	-0.401178	0.125815	-2.568

Table 6: Coeficientes del spline cúbico condicionado para la Curva 2

i	x_i	a_i	b_i	c_i	d_i
0	27.7	4.1	0.33	2.26205	-3.79941
1	28	4.3	0.661386	-1.15743	0.29604
2	29	4.1	-0.765347	-0.269307	-0.0653465

Table 7: Coeficientes del spline cúbico condicionado para la Curva 3

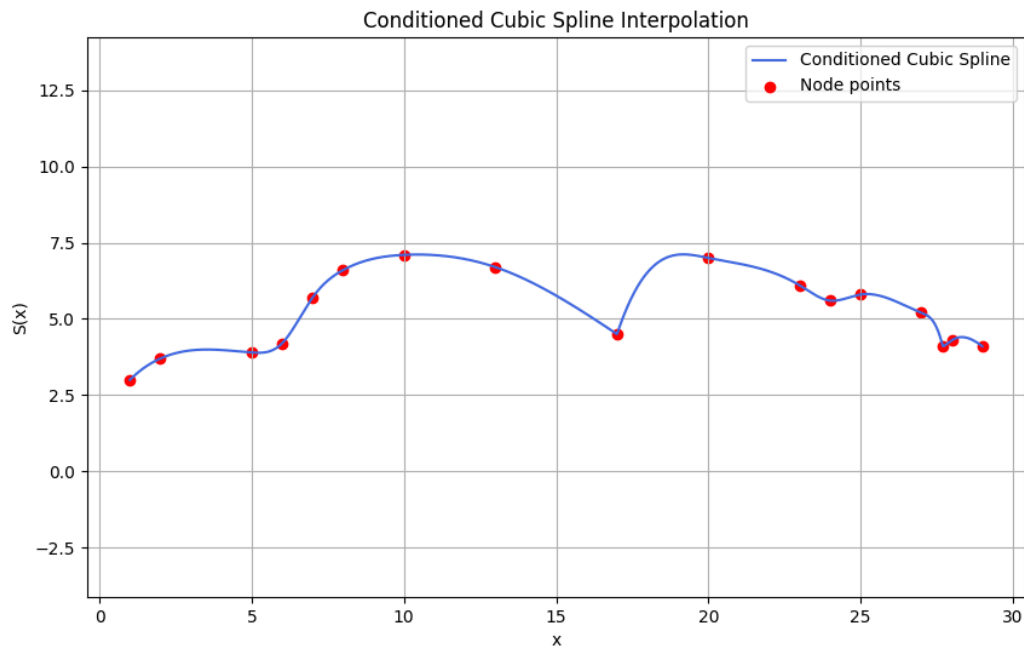


Figure 3: Contorno de Snoopy usando splines cúbicos